

동작 원리를 끝까지 이해하고, 이해한 것을 다시 사용할 수 있도록 남기는 엔지니어입니다.

명확히 설명되어 있지 않은 동작도 코드와 실험, 비교 검증을 통해 파악하고 필요한 기준과 구조를 만들어갑니다.

파악한 내용은 개인의 지식으로만 두지 않고, 모듈과 가이드, 문서로 옮겨 함께 사용할 수 있게 정리합니다.

가족이 쓰는 서비스를 홈 서버에서 직접 운영하며 시스템을 만드는 일뿐 아니라 안정적으로 유지하는 방법도 함께 배우고 있습니다.

EMAIL canon1107@naver.com

PHONE 010.2089.6967

Experience

티맥스 티베로

데이터베이스 엔진 연구원

2024.08 - 현재

Optimizer 기능 개발과 이슈 분석, Oracle 호환 기능 구현을 수행했고, Jam 기반 빌드 시스템을 CMake로 전환했습니다.
현재는 대규모 C 코드베이스의 코드 품질을 높이기 위한 연구를 진행하고 있습니다.

주식회사 일상도시

도시재생 분야 연구

2020.08 - 2023.05

동인천역 주변 역사문화자산 발굴 및 아카이빙, 도시재생활성화계획 수립 용역을 수행했습니다.

Skills

Language · DBMS

- C
- SQL
- Tibero
- Oracle

Build · Infra

- CMake
- Linux
- Docker
- Shell

Education

삼성 청년 SW 아카데미 10기

2023.06 - 2024.05

1학기 성적우수상 · 1학기 관통 프로젝트 최우수상 · 2학기 특화 프로젝트 우수상

성균관대학교 건축학과

2016.03 - 2022.02

학사 졸업

자격

2023 - 2024

정보처리기사 · SQL 개발자

Projects

Oracle 호환 기능 구현, SQL 실행계획 이슈 분석, 빌드 시스템 전환처럼 DBMS 개발과 운영 기반을 연결하는 경험을 중심으로 정리했습니다.

티맥스 티베로

2025.08 - 2026.04

DBMS 빌드 시스템 Jam → CMake 전환

분석

설계

구현

문서화되지 않은 Jam 기반 빌드 규칙을 소스코드와 기존 빌드 흐름을 기반으로 역분석하고, CMake 기반 구조로 전환했습니다.

배경

- 오래 유지된 Jam 기반 빌드 시스템은 규칙이 여러 파일과 개인 경험에 흩어져 있어, 신규 작성과 수정이 쉽지 않았습니다.
- 빌드 파일 간 의존성 누락으로 병렬 빌드 수준을 높이기 어려웠고, 개발 중 한 번에 빌드가 끝나지 않아 재시도가 필요했습니다.

작업 내용

- 소스코드와 기존 빌드 흐름을 기준으로 Jam 규칙을 역분석하고, 모듈별 빌드·링크 관계를 정리했습니다.
- CMake 빌드 시스템을 설계·구현하며 의존성을 명시적으로 선언하는 구조로 재편했습니다.
- Linux / Windows / AIX / Solaris 플랫폼별 빌드·링크 이슈를 대응했습니다.
- 빌드 파일 작성 방식과 사용법을 가이드 문서로 정리하고 전사 세미나로 공유했습니다.
- 전환 후 연구원들의 문의와 개선 요청에 대응하며 빌드 시스템을 안정화했습니다.

성과

- 동일 형상 기준 빌드 시간을 약 34~38분에서 24~26분으로 줄여 **약 30% 개선**했습니다.
- 의존성 누락으로 인한 빌드 실패를 줄이고, **재시도 없이 한 번에 빌드되는 환경**을 마련했습니다.
- 메인 개발 브랜치 적용 후 다른 버전 브랜치 전환 요청을 받아 확대 적용했습니다.
- 처음 접하는 사람도 문서만 보고 빌드 파일을 작성할 수 있도록 기준과 예시를 남겼습니다.

Optimizer Pending Statistics 기능 개발

스펙 분석

설계

구현

통계 수집 시점과 실행계획 반영 시점을 분리하는 Oracle 기능을 Tibero에 신규 개발했습니다.

배경

- 운영 DB에서는 통계를 수집하는 순간 실행계획이 바뀔 수 있어, 통계 갱신 자체가 성능 변동 리스크가 됩니다.
- 통계를 미리 수집해두고 검증 후 반영할 수 있는 Oracle의 Pending Statistics 기능을 고객사가 요청했습니다.

작업 내용

- 상세 스펙이 공개되지 않아 Oracle 공식 문서, dictionary view, 실험 결과를 바탕으로 동작을 역추적했습니다.
- 보류 통계의 수집·저장·조화·반영 로직을 기존 Tibero 엔진 구조에 맞게 설계·구현했습니다.
- table / index / column / partition 단위 테스트 시나리오를 구성하고 Oracle과 실행계획을 비교 검증했습니다.
- 보류 통계가 없을 때 Optimizer가 의도와 다르게 Dynamic Sampling을 사용하는 경우를 발견하고 기존 통계를 사용하도록 설계를 수정했습니다.

성과

- Oracle 동작을 역분석해 Tibero 엔진 구조에 맞게 이식하고, 실행계획 비교로 기능 신뢰성을 검증했습니다.
- 고객사가 통계 갱신에 따른 성능 변동 리스크 없이, **통계를 미리 검증한 뒤 반영할 수 있는 환경**을 확보했습니다.

Optimizer 모듈 이슈 분석 및 대응

재현

원인 분석

수정

내부 테스트 및 고객사 환경에서 발생한 Optimizer 관련 이슈를 재현하고 원인을 분석해 대응했습니다.

배경

- Optimizer 이슈는 성능 저하, 서버 크래시, 결과 오류처럼 증상이 다양하고 SQL 조건에 따라 재현 여부가 달라집니다.
- 수정이 다른 쿼리의 실행계획에도 영향을 줄 수 있어, 최소 재현과 영향 범위 확인이 중요했습니다.

작업 내용

- DB 버전 업그레이드 후 특정 SQL 성능이 저하된 이슈에서 SQL Trace와 실행계획을 비교해 Index Scan이 Full Scan으로 바뀐 지점을 확인했습니다.
- 특정 집계 쿼리 서버 크래시 이슈에서 재현 쿼리를 최소 케이스로 축소하고 정상·비정상 케이스를 비교했습니다.
- 병렬 쿼리 결과 오류에서 병렬 실행 시 데이터 분배 과정의 중복 문제를 분석해 수정했습니다.
- TPC-H 계열 쿼리와 SQL Plan 이슈를 분석하며 쿼리 변환과 실행계획 생성 흐름을 추적했습니다.

성과

- 성능 저하, 크래시, 결과 오류 등 유형이 다른 이슈를 **재현부터 수정까지 담당**했습니다.
- 이슈별 재현 조건과 분석 과정을 문서로 정리해 유사 이슈 대응에 활용했습니다.

Header Dependency / Include Cleanup 기반 조사

대형 C 코드베이스

모듈화 기반

대형 C 코드베이스의 헤더 의존성을 줄이고 모듈화 가능한 개발 기반을 만들기 위해 진행 중인 조사입니다.

작업 내용

- 헤더 유형을 분류하고, self-contained header 여부를 확인할 수 있는 검사 기준을 정리하고 있습니다.
- compile_commands 기반 분석 가능성을 검토하고 IWYU, clang-tidy, cppcheck의 적용 범위와 한계를 비교하고 있습니다.
- Linux뿐 아니라 Windows / AIX / Solaris 환경에서 적용할 때의 제약을 함께 확인하고 있습니다.

진행 방향

- include cleanup을 개인 감각이 아니라 재현 가능한 규칙과 도구 기반으로 진행하기 위한 기반을 마련하고 있습니다.
- 빌드 시스템 전환 이후의 개발 생산성 병목을 구조적으로 줄이는 방법론으로 확장하는 것이 목표입니다.

홈 서버 구축 및 운영

구축

운영

개선

사진 백업·조회, 문서 관리, Git 등 가족과 개인이 사용하는 서비스를 직접 구축해 운영하는 홈 서버 인프라입니다.

배경

- 가족 사진을 안전하게 모아두고, 외부에서도 조회할 수 있는 개인 서비스 환경이 필요했습니다.
- 개인 개발과 기술 테스트를 반복할 수 있는 독립적인 서버 환경도 함께 만들고자 했습니다.

작업 내용

- OPNsense 방화벽을 두고 용도별 IP 대역과 포트 정책을 문서화해 관리했습니다.
- Caddy Reverse Proxy와 Cloudflare로 외부 접근 경로를 단일화하고, Authentik으로 서비스 로그인을 통합했습니다.
- Immich, 노트, Git 등 필요한 서비스를 Docker 기반으로 추가·삭제할 수 있게 구성했습니다.
- iCloud → NAS → 외장하드 백업 흐름을 만들고, 서버와 컨테이너 상태를 확인할 수 있는 모니터링 흐름을 구성했습니다.

운영 개선

- 디스크 용량 부족으로 인증 서비스가 멈춘 경험을 바탕으로, 장애가 다른 서비스로 전파되지 않도록 모니터링 항목을 보강했습니다.
- 테스트용 VM을 분리해 실험 중 실수가 실제 사용 중인 서비스에 영향을 주지 않도록 운영 방식을 개선했습니다.
- 개인 서비스라도 인증, 백업, 모니터링, 장애 전파를 함께 고려하며 운영하고 있습니다.

SSAFY

2024.04 - 2024.05

Omegi : 분산 시스템 오류 추적 서비스

Python 자동 계측 라이브러리

로그 수집 파이프라인

GitHub ↗

MSA 환경에서 여러 서비스에 걸친 오류 흐름을 추적하고 기록하는 서비스입니다.

배경

- MSA 구조에서는 하나의 요청이 여러 서비스를 거치기 때문에, 오류가 발생한 지점과 호출 흐름을 한눈에 파악하기 어렵습니다.
- Java와 Python 서비스가 함께 있을 때 trace가 끊기는 문제가 있어 공통 전파 형식이 필요했습니다.

작업 내용

- OpenTelemetry 기반 함수 단위 자동 계측 라이브러리를 개발하고 **PyPI에 배포**했습니다.
- Decorator로 Span을 생성해 함수 호출 간 부모-자식 관계를 유지하고, Kafka로 trace를 전송하는 Exporter를 구현했습니다.
- Java-Python 서비스 간 trace가 이어지지 않는 문제를 W3C Trace Context로 전파 형식을 통일해 해결했습니다.
- Kafka → Redis 작업 큐 → Elasticsearch 흐름으로 순서 없이 도착하는 로그를 하나의 trace로 모아 가공했습니다.

SSAFY

2024.02 - 2024.04

Englising : 팝송 기반 영어 학습 게임

데이터 파이프라인

게임 API

배포 환경

GitHub ↗

팝송 가사 빈칸 맞추기로 영어를 학습하는 웹 서비스입니다.

작업 내용

- Spotify API, 가사, YouTube 데이터를 수집하고 게임 출제에 사용할 수 있는 형태로 가공했습니다.
- Queue → Thread Worker → Docker 컨테이너 분리 구조로 파이프라인을 개선했습니다.
- WebSocket 기반 멀티플레이 진행 로직, Redis 기반 게임 상태 저장, Docker-Jenkins-Nginx 배포 환경을 구성했습니다.

SSAFY

2024.01 - 2024.02

Togeduck : 생일카페 지도 Android 앱

팀장

Android 화면 개발

실시간 기능

GitHub ↗

생일카페 정보를 지도로 탐색하고 실시간으로 소통하는 Android 앱입니다.

작업 내용

- Kotlin 기반 Android 주요 화면과 지도 위 생일카페 마커 표시·클러스터링 기능을 개발했습니다.
- WebSocket 기반 사용자 실시간 위치 공유와 채팅 기능을 구현했습니다.
- 사용자별 위치 좌표를 관리해 잦은 마커 갱신으로 발생하는 오류에 대응했습니다.